

Once we are done with this, it's time to collect user input as a prompt and conclude with output. To achieve this, we will declare another function called `'chat_with_bot()'` and work with a `while` loop to make the condition work in a loop, once the result is shared with the user. To do that, we declare a loop with the syntax of `'while True:'` and inside the `'chat_with_bot()'` function is a Python function that implements a simple chatbot. It takes user input, generates a response using the OpenAI Codex API, and prints the response to the console. The chatbot continues to interact with the user until the user enters `'exit'`.

The function starts by printing a greeting message to the console. It enters a `while` loop that continues until the user enters `'exit'`. Inside the loop, it prompts the user for input using the `input` function. If the user enters `'exit'`, the function prints a goodbye message and breaks out of the loop. Otherwise, it constructs a prompt by appending the user input to the string `'User: '` and assigns it to the variable `'prompt'`. It calls the `'generate_response'` function, passing the prompt as an argument, to generate a response using the OpenAI Codex API. The response is extracted from the API response object and assigned to the variable `response`. The response is printed to the console with the prefix `'Chatbot: '`.

The function code block is:

```
def chat_with_bot():
    print("Chatbot: Hi there! Ask me
    anything or just chat with me.")
    while True:
        user_input = input("You: ")
        if user_input.lower() == 'exit':
            print("Chatbot: Goodbye!")
            break
        prompt = f"User: {user_input}\n
        Chatbot:"
        response = generate_
        response(prompt)
        print(f"Chatbot: {response}")
```

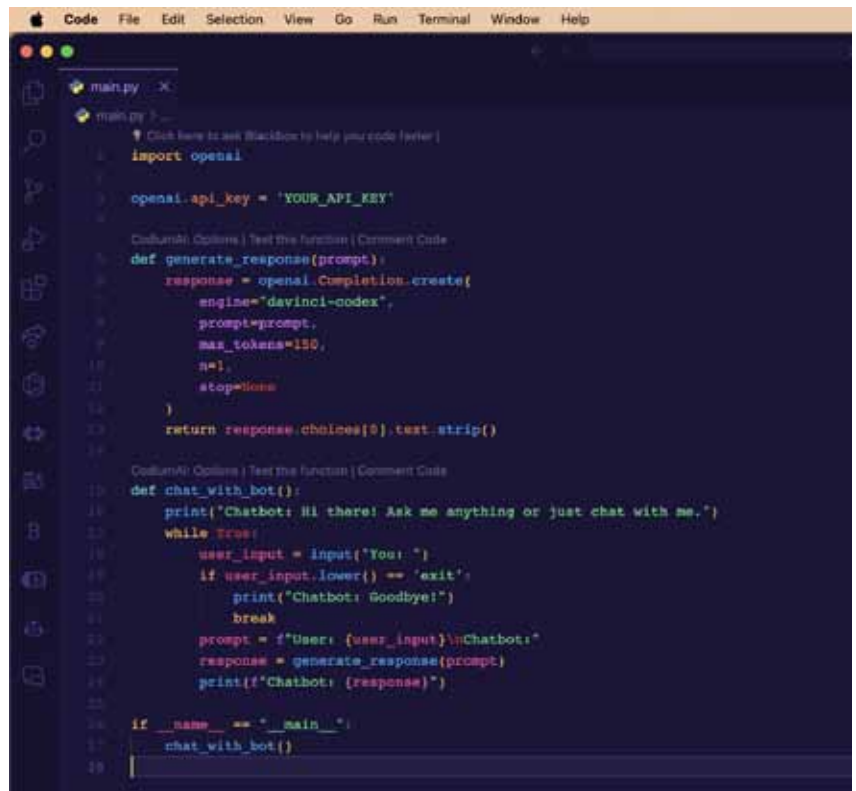


Figure 3: Complete code block

To sum up, your complete code block should look somewhat similar to the code screenshot given in Figure 3, and the code should not show any syntax error. If everything works well, we are ready to fetch the response from the chatbot. So we will run this bot by typing the command `'python3 main.py'` (sometimes `'python bot.py'` or `'py bot.py'` works as well). Hence, please make sure Python is available on your system. If you have done everything correctly, you will get an output of a greeting message. Please remember this is just a basic example of how you can connect OpenAI's GPT-3 code to build a chatbot.

Important notes

- Replace `'YOUR_API_KEY'` with the actual API key you received

from OpenAI.

- Experiment with different prompts, engines, and parameters to achieve the desired chatbot behaviour.
- Be mindful of OpenAI's use-case policies and ethical guidelines when creating and deploying chatbots.

Keep in mind that developments are continuous; so it's a good idea to refer to the latest OpenAI documentation for any changes or additional features.

The power of generative AI, exemplified by GPT-3, has far-reaching implications for technology, business, and society. As we look forward to future iterations, collaborative efforts and ethical considerations will shape the transformative impact of generative AI on our world. **END** 🐧

By: S.V. Rohith

The author specialises in RPA, Python, and Web 3.0, and has over five years of expertise in creating and managing Telegram bots.